

はじめてのメディアアート制作

プログラムで図形や色を動かし，自分だけのアート表現を体験

湘南工科大学 情報学部 情報学科

情報メディア専攻

清水 哲也

今日のゴール

今日は，次の3つを体験します．

1. p5.js Web Editorを開く
2. コードをコピー＆ペーストして動かす
3. 色，形，動きを変えて，自分だけの作品にする

プログラミングが初めてでも問題ありません．

今日は「全部理解する」よりも，動く作品を作ること为目标にします．

今日つくるもの

動くメディアアート

- 図形を描く
- 色を変える
- マウスに反応させる
- 動きをつける
- 自分の好みに改造する

最後には，自分だけの動くアート作品を作ります．

今日の約束

コピペしてよいです

プログラムは，最初からすべて入力しなくてよいです。
まずはコピー＆ペーストして，動く状態を作ります。

そのあとで，

- 数字を変える
- 色を変える
- 図形を変える
- 動きを変える

という順番で改造します。

今日の約束

ログインはしません

p5.js Web Editorはログインすると保存できますが、今日はログインしません。

理由：

- メールアドレスが必要になる
- アカウント作成で時間がかかる
- 今日は作品づくりを優先する

気に入った作品は、最後にスクリーンショットで残します。

p5.js Web Editorを開く

ブラウザで次のURLを開きます.

<https://editor.p5js.org/>

開いたら, 左側にコードを書く場所, 右側に実行結果が表示されます.



☒ 自動更新

Thoughtful wavelength

p5.js 1.11.13



>

sketch.js

プレビュー

```

1▼ function setup() {
2  createCanvas(400, 400);
3 }
4
5▼ function draw() {
6  background(220);
7 }

```

コンソール

クリア ▼

画面の見方

- 左側：プログラムを書く場所
- 右側：作品が表示される場所
-  ボタン：プログラムを実行する
- ■ボタン：プログラムを止める



p5.js 1.11.13



> sketch.js

プレビュー

```
1 function setup() {  
2   createCanvas(400, 400);  
3 }  
4  
5 function draw() {  
6   background(220);  
7 }
```

プログラムを書く場所

作品が表示される場所

コンソール

クリア ▼

まず動かしてみよう

最初からプログラムが書かれているので実行ボタン（



ボタン）を押して実行してみましょう

最初に書かれているプログラム

```
function setup() {  
  createCanvas(400, 400);  
}  
function draw() {  
  background(220);  
}
```

少しプログラムを追加して見ましょう

```
function setup() {  
  createCanvas(400, 400);  
  noStroke();  
}  
  
function draw() {  
  background(220);  
  
  fill(255, 120, 120);  
  circle(200, 200, 120);  
}
```

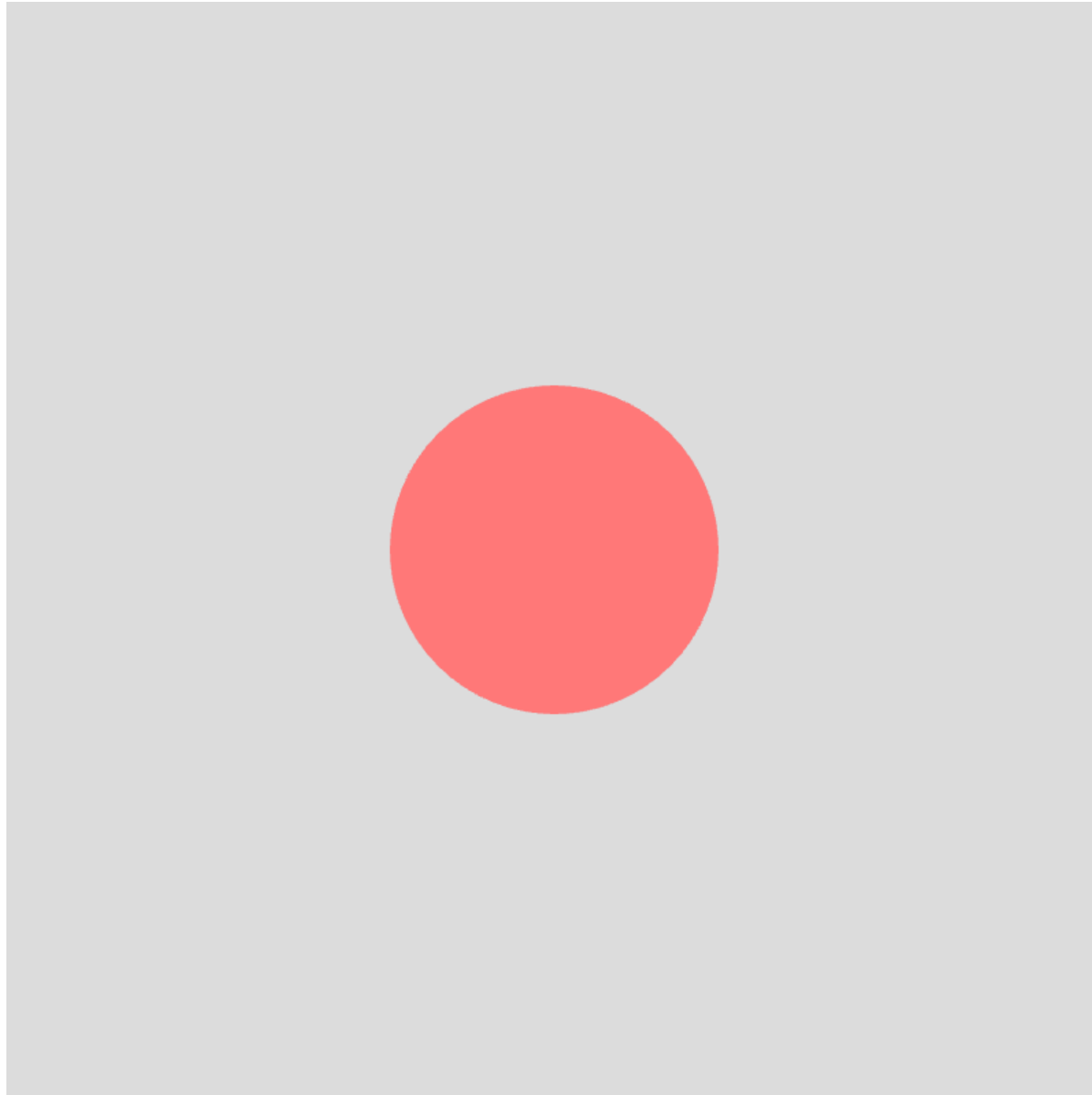
何が起きた？

このプログラムでは,

- 400×400の画面を作る
- 背景を明るいグレーにする
- 中央に円を描く

という処理をしています.

まずは, 円が表示されれば成功です.



p5.jsの基本形

p5.jsでは，主に2つの場所を使います．

```
function setup() {  
  // 最初に1回だけ実行される  
}  
function draw() {  
  // 何度も繰り返し実行される  
}
```

今日は細かい文法よりも，どこを変えると作品が変わるかを見ます．

setup() と draw()

名前	役割
setup()	最初の準備を書く場所
draw()	何度も実行される場所

draw() が何度も実行されるため、動きのある作品を作ることができます。

作品を変えるポイント

さきほどのコードでは，主にここを変えます．

```
background(240);  
fill(255, 120, 120);  
circle(200, 200, 120);
```

命令	変わるもの
background()	背景色
fill()	図形の色
circle()	円の位置と大きさ

色を変えてみよう

`fill()` の中の数字を変えます.

```
fill(255, 120, 120);
```

数字は, 赤・緑・青の強さを表します.

```
fill(赤, 緑, 青);
```

数字は `0~255` の範囲で指定します.

色の例

```
fill(255, 0, 0);    // 赤  
fill(0, 150, 255);  // 青  
fill(255, 200, 0);  // 黄色  
fill(150, 80, 255); // 紫  
fill(0, 220, 160);  // 緑
```

好きな色に変えてみましょう.

大きさを変えてみよう

`circle()` の最後の数字を変えます.

```
circle(200, 200, 120);  
circle(x座標, y座標, 直径);
```

例：

```
circle(200, 200, 60);    // 小さい円  
circle(200, 200, 180);   // 大きい円
```

位置を変えてみよう

circle() の最初の2つの数字を変えます.

```
circle(200, 200, 120);  
circle(x座標, y座標, 直径);
```

- x座標：右に行くほど大きくなる
- y座標：下に行くほど大きくなる

練習1：自分の色にする

次の3つを変えてください.

1. 背景色
2. 円の色
3. 円の大きさ

変更する場所：

```
background(240);  
fill(255, 120, 120);  
circle(200, 200, 120);
```

マウスに反応させよう

次は、円がマウスについてくる作品を作ります.

先ほどのプログラムの `circle` の1つ目と2つ目を変更しましょう.

```
function setup() {  
  createCanvas(400, 400);  
  noStroke();  
}  
function draw() {  
  background(240);  
  fill(0, 150, 255);  
  circle(mouseX, mouseY, 80);  
}
```

mouseX と mouseY

mouseX と mouseY は、マウスの位置を表します。

```
circle(mouseX, mouseY, 80);
```

- mouseX : マウスの横の位置
- mouseY : マウスの縦の位置

マウスを動かすと、円も一緒に動きます。

色もマウスで変えてみよう

次は, `fill` の1つ目と2つ目を変更してみましょう.

```
function setup() {  
  createCanvas(400, 400);  
  noStroke();  
}  
function draw() {  
  background(240);  
  fill(mouseX, mouseY, 200);  
  circle(200, 200, 120);  
}
```

マウスを動かすと, 円の色が変わります.

マウスで大きさを変えてみよう

```
function setup() {  
  createCanvas(400, 400);  
  noStroke();  
}  
function draw() {  
  background(240);  
  fill(255, 140, 0);  
  circle(200, 200, mouseX);  
}
```

マウスを右に動かすほど、円が大きくなります。

今日のメイン作品

動く円＋マウスで色が変わる作品

```
let x = 100;
let speed = 3;
function setup() {
  createCanvas(400, 400);
  noStroke();
}
function draw() {
  background(240);
  fill(mouseX, 120, 255);
  circle(x, mouseY, 90);
  x = x + speed;
  if (x > width || x < 0) {
    speed = speed * -1;
  }
}
```

メイン作品のポイント

この作品では,

- 円が左右に動く
- マウスの上下で円の高さが変わる
- マウスの横位置で色が変わる
- 画面の端で跳ね返る

という動きが入っています.

ここまでできれば, 今日のゴール達成です.

改造ポイント

次の場所を変えると，作品の印象が大きく変わります．

```
let speed = 3;  
background(240);  
fill(mouseX, 120, 255);  
circle(x, mouseY, 90);
```

場所	変わるもの
speed	動く速さ
background	背景色
fill	円の色
90	円の大きさ

改造例1：速くする

```
let speed = 8;
```

数字が大きいほど速く動きます。

```
let speed = 1;  // ゆっくり  
let speed = 3;  // ふつう  
let speed = 8;  // 速い
```

改造例2：背景を暗くする

```
background(20);
```

背景を暗くすると，光る作品のように見えます．

```
background(0);    // 黒  
background(20);   // 暗いグレー  
background(240);  // 明るいグレー
```

改造例3：円を大きくする

```
circle(x, mouseY, 150);
```

最後の数字が直径です。

```
circle(x, mouseY, 50);    // 小さい  
circle(x, mouseY, 90);    // ふつう  
circle(x, mouseY, 150);   // 大きい
```

軌跡を残してみよう

背景を毎回描かないと，前の図形が残ります。
これを使うと，光の軌跡のような表現になります。

```
let x = 100;
let speed = 3;
function setup() {
  createCanvas(400, 400);
  background(0);
  noStroke();
}
function draw() {
  fill(mouseX, mouseY, 255, 40);
  circle(x, mouseY, 80);
  x = x + speed;
  if (x > width || x < 0) {
    speed = speed * -1;
  }
}
```


透明度を使う

`fill()` に4つ目の数字を入れると、透明度を指定できます。

```
fill(0, 150, 255, 40);  
fill(赤, 緑, 青, 透明度);
```

- 0：透明
- 255：不透明
- 40：かなり薄い

自由制作

ここからは、自分の作品にします。

変更してよいもの：

- 背景色
- 図形の色
- 図形の大きさ
- 動く速さ
- マウスへの反応
- 軌跡の残り方

正解はありません。

見た目がおもしろければ成功です。

困ったとき

画面が真っ白になったら

まず確認すること：

- `;` が抜けていないか
- `{` と `}` の数が合っているか
- `circle` や `background` のつづりが違っていないか
-  ボタンを押したか

わからないときは，補助学生に声をかけてください。

困ったとき

よくあるミス

```
background(240)    // ; がない  
fill(255, 0, 0;    // ) がない  
circl(200, 200, 100); // circle のつづりが違う
```

プログラムは、1文字違うだけで止まることがあります。
ただし、直せばまた動きます。

作品の残し方

今日はログインしないため、保存は必須にしません。

残したい場合：

1. 作品を表示する
2. スマホで撮る
3. 必要ならコードもスマホで撮る

自宅で続けたい人は、付録のログイン方法を見てください。

まとめ

今日体験したこと：

- プログラムで図形を描く
- 色を変える
- 図形を動かす
- マウスに反応させる
- 自分だけの作品に改造する

プログラミングは、計算だけでなく、表現の道具にもなります。

情報学科情報メディア専攻で学べること

情報メディア専攻では、プログラミングを使って、

- Webアプリ
- ゲーム
- CG
- VR・AR
- メディアアート
- インタラクティブ作品

などを作ることができます。

付録A：p5.js Web Editorにログインするには

自宅で作品を保存したい場合は、ログインします。

1. <https://editor.p5js.org/> を開く
2. 右上の「**Sign up**」を押す
3. メールアドレスなどを入力する
4. ログイン後、「File」→「Save」で保存する

※今日は授業時間を優先するため、ログインは行いません。

付録B：光の軌跡

```
let x = 0;
let speed = 4;
function setup() {
  createCanvas(400, 400);
  background(0);
  noStroke();
}
function draw() {
  fill(mouseX, 100, 255, 35);
  circle(x, mouseY, 70);
  x = x + speed;
  if (x > width || x < 0) {
    speed = speed * -1;
  }
}
```

付録C：カラフルなシャボン玉

```
function setup() {  
  createCanvas(400, 400);  
  noStroke();  
}  
function draw() {  
  background(245);  
  fill(255, 100, 150, 120);  
  circle(mouseX, mouseY, 120);  
  fill(100, 200, 255, 120);  
  circle(400 - mouseX, mouseY, 90);  
  fill(255, 220, 80, 120);  
  circle(mouseX, 400 - mouseY, 70);  
}
```

付録D：ミラーボール

```
function setup() {  
  createCanvas(400, 400);  
  noStroke();  
}  
function draw() {  
  background(20);  
  fill(mouseX, 100, 255);  
  circle(mouseX, mouseY, 80);  
  fill(255, mouseY, 100);  
  circle(400 - mouseX, mouseY, 80);  
  fill(100, 255, mouseX);  
  circle(mouseX, 400 - mouseY, 80);  
  fill(255, 255, 100);  
  circle(400 - mouseX, 400 - mouseY, 80);  
}
```

付録E：流れ星

```
let x = 0;
let y = 80;
let speed = 5;
function setup() {
  createCanvas(400, 400);
  noStroke();
}
function draw() {
  background(10, 10, 30);
  fill(255, 255, 180);
  circle(x, y, 30);
  fill(255, 255, 180, 80);
  circle(x - 30, y - 15, 20);
  circle(x - 60, y - 30, 10);
  x = x + speed;
  y = y + 1;
  if (x > width + 60) {
    x = -60;
    y = random(50, 200);
  }
}
```

付録F：雨のような線

```
function setup() {  
  createCanvas(400, 400);  
  background(20, 30, 60);  
}  
function draw() {  
  stroke(100, 200, 255, 80);  
  line(mouseX, 0, mouseX, height);  
  stroke(255, 255, 255, 40);  
  line(0, mouseY, width, mouseY);  
}
```

付録G：ゆらぐ四角

```
let t = 0;
function setup() {
  createCanvas(400, 400);
  rectMode(CENTER);
  noStroke();
}
function draw() {
  background(240);
  fill(mouseX, 120, 220);
  rect(200, 200, 100 + sin(t) * 80, 100 + cos(t) * 80);
  t = t + 0.05;
}
```

付録H：ドットペイント

```
function setup() {  
  createCanvas(400, 400);  
  background(255);  
  noStroke();  
}  
function draw() {  
  fill(mouseX, mouseY, 200, 80);  
  circle(mouseX, mouseY, 30);  
}
```

付録I：クリックで画面を消す

```
function setup() {  
  createCanvas(400, 400);  
  background(255);  
  noStroke();  
}  
function draw() {  
  fill(mouseX, mouseY, 255, 60);  
  circle(mouseX, mouseY, 40);  
}  
function mousePressed() {  
  background(255);  
}
```


付録J：改造のヒント

変えたいもの	変更する場所
背景色	<code>background()</code>
図形の色	<code>fill()</code>
線の色	<code>stroke()</code>
円の大きさ	<code>circle()</code> の3つ目
四角の大きさ	<code>rect()</code> の3つ目・4つ目
動く速さ	<code>speed</code> の数字
マウス反応	<code>mouseX</code> , <code>mouseY</code>

付録K：色の組み合わせ例

```
fill(255, 80, 80);    // 赤系  
fill(80, 160, 255);   // 青系  
fill(255, 210, 80);   // 黄系  
fill(120, 255, 180);  // 緑系  
fill(180, 120, 255);  // 紫系  
fill(255, 120, 200);  // ピンク系
```

色を決めるだけでも，作品の印象は大きく変わります。

付録L：自宅で続ける人へ

p5.jsはブラウザだけで使えます。

<https://editor.p5js.org/>

自宅で続ける場合は、

- ログインして保存する
- 今日のコードをもう一度貼り付ける
- 数字や色を変えて試す

という流れで制作できます。